

**DESIGN AND SIMULATE A SMART FLOOD LEVEL MONITORING
SYSTEM USING ULTRASONIC SENSOR. THE SYSTEM
CONTINUOUSLY MONITORS WATER LEVELS AND PROVIDES
ALERTS DURING CRITICAL FLOOD CONDITIONS FOR DISASTER
MANAGEMENT APPLICATIONS.**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B	– 24MER030
MOHAMMED SABEER S	– 24MER031
PRAAVIN RAJAA G	– 24MER039
PRASANNA T	– 24MER042
DEVA PRIYAN R	– 24MEL064
SATHISH KUMAR K	– 24MEL067

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Smart Flood Level Monitoring System is an embedded, sensor-based early-warning solution designed to continuously track water levels in rivers, canals, dams, or urban drainage channels and automatically alert relevant authorities and residents when flood-risk thresholds are crossed. The system combines a non-contact ultrasonic distance sensor, a microcontroller-based control unit, local visual/audible indicators, and a GSM/IoT communication module into a single, self-contained monitoring station that can operate with minimal human supervision.

Core idea: an ultrasonic sensor (HC-SR04) is mounted at a fixed, known height above the water body — typically on a bridge, pole, or gantry. It continuously measures the distance to the water surface below. Since the sensor's mounting height is fixed and known, this distance reading directly translates into the current water level:

$$\text{Water Level} = \text{Sensor Mounting Height} - \text{Measured Distance}$$

The microcontroller samples this reading at regular intervals (e.g., every 1–2 seconds), converts it into a percentage of the channel's total depth, and continuously compares it against four predefined threshold zones — **Safe, Watch, Danger, and Critical** — to determine the current flood risk level.

Key components of the system:

Module	Role
Ultrasonic sensor (HC-SR04)	Measures real-time distance to water surface
Microcontroller (Arduino/NodeMCU)	Processes readings, runs threshold logic, controls outputs
LCD display	Shows live water level, depth, and status locally
Buzzer & LED indicators	Provide immediate on-site visual/audible alerts
GSM/WiFi module (SIM800L / ESP8266)	sends SMS or cloud alerts to authorities and the public

What the system does:

- Continuously monitors water level without physical contact with the water (avoiding corrosion, biofouling, and debris damage common to float or conductivity sensors).
- Classifies the current condition into graded risk zones rather than a simple binary "flood/no flood" state, enabling a proportionate response.
- Triggers local alarms (LEDs, buzzer, siren) instantly for people near the site.
- Sends automated SMS/GSM alerts to disaster management authorities and registered residents only when the risk zone changes — avoiding alert fatigue from repeated notifications.
- Logs and displays real-time data (distance, depth, percentage level, rate of rise) for on-site monitoring and diagnostics.

2. Problem Statement

Floods remain one of the most frequent and damaging natural disasters worldwide, and their impact is often worsened not by the flood itself but by the delay in detecting and responding to rising water levels. Existing flood monitoring practices in many regions — especially in smaller towns, rural areas, and along minor rivers and canals — still rely heavily on manual observation, periodic gauge readings taken by personnel, or outdated infrastructure that was never designed for real-time, automated response. Therefore, an automated system is required to monitor soil conditions and provide water only when it is necessary.

This creates several critical gaps:

- Delayed detection — Manual monitoring depends on personnel physically visiting a site or checking a gauge at fixed intervals, which means water level changes between checks go unnoticed. During fast-onset events like flash floods, this delay can be fatal.
- Lack of continuous, real-time data — Without continuous sensing, disaster management authorities have no way of tracking the *rate* at which water is rising, which is often more important than the absolute level itself for predicting how much time remains before critical flooding occurs.
- Delayed or absent communication to authorities — Even when a dangerous water level is observed, alerting the relevant disaster response teams often depends on manual phone calls or radio communication, introducing further delay and human error into the response chain.
- Unsuitability of contact-based sensors — Conventional float switches or conductivity-based level sensors sit in the water and are prone to fouling from debris, sediment, and corrosion over time, leading to unreliable readings or complete sensor failure exactly when they are needed most — during a flood event.
- No graded or proportionate response — Many existing systems, where automated at all, only provide a binary "flood" or "no flood" signal, giving no early indication that conditions are worsening and no opportunity for a graded, staged response (e.g., issuing a preliminary watch before escalating to a full evacuation alert).
- Poor scalability — Manual and semi-automated systems are difficult and expensive to replicate across the many points along a river, canal, or drainage network that would need to be monitored to build an accurate, region-wide picture of flood risk.

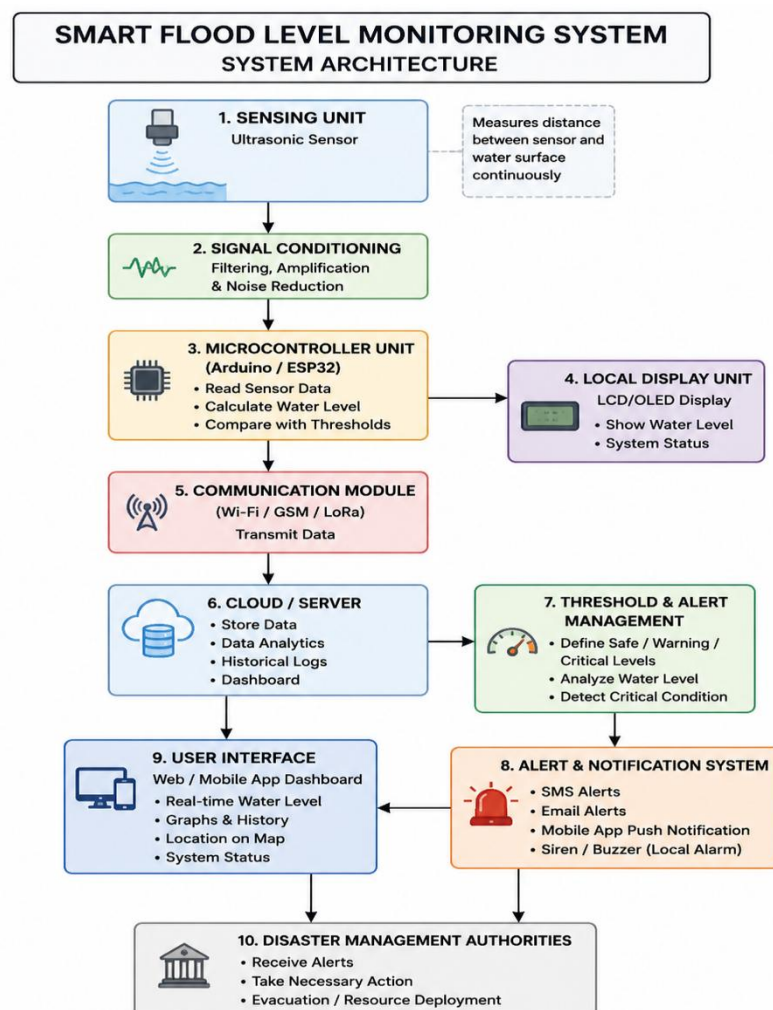
3. Proposed Solution

To address the delays and unreliability of manual and contact-based flood monitoring, this project proposes an automated, non-contact, ultrasonic-sensor-based Smart Flood Level Monitoring System. An HC-SR04 ultrasonic sensor mounted at a fixed height above the channel continuously measures the distance to the water surface using reflected sound pulses, eliminating the need for any sensor component to touch or submerge in the water and avoiding the corrosion and fouling issues common to float or conductivity sensors. A microcontroller (Arduino/NodeMCU) samples this reading every one to two seconds, converts it into a water

level and depth percentage, filters out noise from surface ripples, and calculates the rate of rise between samples to give early warning of how quickly conditions are worsening. This processed value is continuously compared against four configurable threshold zones — Safe, Watch, Danger, and Critical — allowing the system to respond proportionately rather than issuing a simple binary flood alert. Locally, the current status is shown on an LCD display while LEDs and a buzzer provide immediate visual and audible warning to anyone nearby, with a relay-driven siren escalating the alarm during critical conditions. Simultaneously, a GSM or WiFi module automatically sends an SMS or cloud notification to disaster management authorities and registered residents the moment the risk zone changes, removing the delay and human error associated with manual phone or radio communication. Because the hardware is low-cost, self-contained, and easily replicated, the same design can be deployed at multiple points along a river, canal, or drainage network, forming a scalable, distributed early-warning system that gives both on-site personnel and remote authorities the real-time information and lead time needed to respond before a flood becomes a disaster.

4. System Architecture

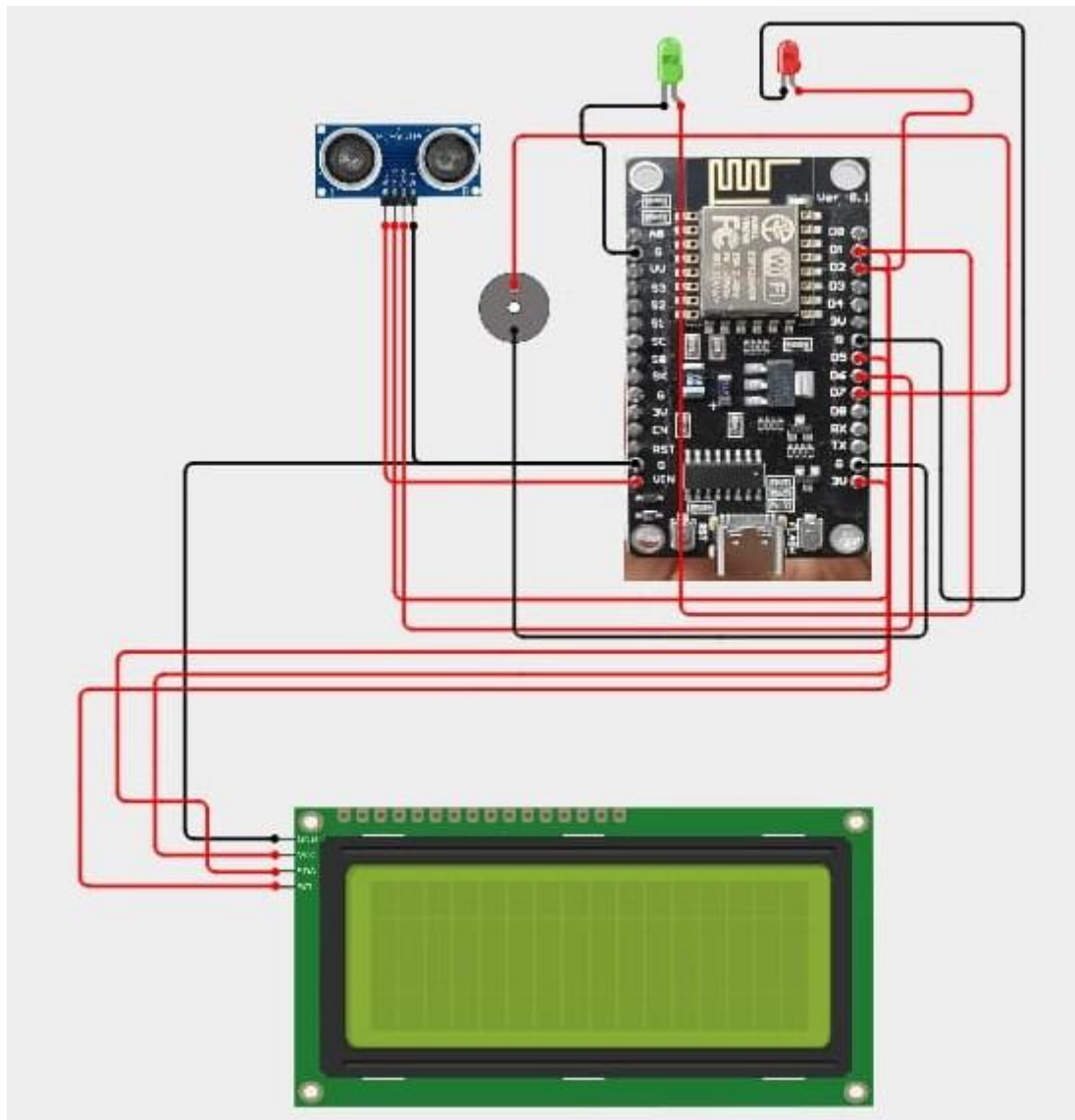
The overall architecture of the system is:



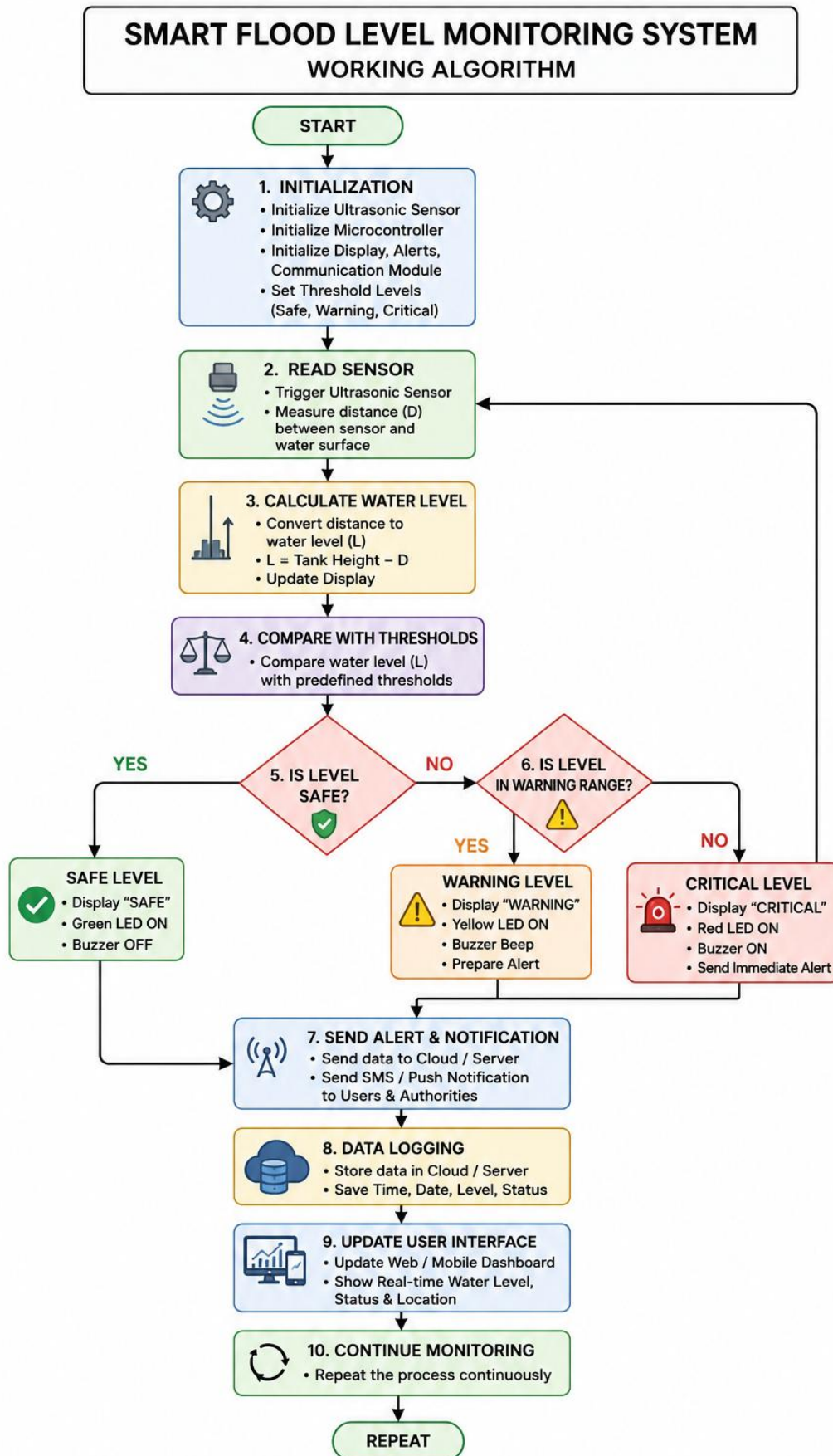
5. Circuit Table

Component	Component pin	Connects to Arduino
HC-SR04 Ultrasonic Sensor	VCC	5V
	GND	GND
	TRIG	Digital pin 9
	ECHO	Digital pin 10
16×2 I2C LCD Display	VCC	5V
	GND	GND
	SDA	A4 (SDA)
	SCL	A5 (SCL)
Buzzer	+ (signal)	Digital pin 8
	– (GND)	GND
Green LED (Safe)	Anode (+)	Digital pin 5
	Cathode (–)	GND
Yellow LED (Watch)	Anode (+)	Digital pin 6
	Cathode (–)	GND
Relay Module (siren/warning light)	IN (signal)	Digital pin 4
	VCC	5V
	GND	GND

6. Circuit Design



7. Algorithm



8. Coding

```
#include <WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

// =====

// WIFI DETAILS

// =====

#define WIFI_SSID  "HITMAN"

#define WIFI_PASSWORD "123456789"

// =====

// ADAFRUIT IO DETAILS

// =====

#define IO_USERNAME "YOUR_ADAFRUIT_USERNAME"

#define IO_KEY  "YOUR_ADAFRUIT_IO_KEY"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);

// =====

// ADAFRUIT IO FEEDS

// =====

AdafruitIO_Feed *waterDistanceFeed = io.feed("water-distance");

AdafruitIO_Feed *waterLevelFeed  = io.feed("water-level");

AdafruitIO_Feed *floodStatusFeed  = io.feed("flood-status");

// =====

// PIN CONNECTIONS

// =====

// Ultrasonic sensor

#define TRIG_PIN 5

#define ECHO_PIN 18
```



```
// LEDs

#define GREEN_LED 25

#define RED_LED 26

// Buzzer

#define BUZZER 27

// LCD

#define SDA_PIN 21

#define SCL_PIN 22

LiquidCrystal_I2C lcd(0x27, 16, 2);

// =====

// FLOOD LEVEL SETTINGS

// =====

// Distance between ultrasonic sensor and water

#define SAFE_DISTANCE 30

#define DANGER_DISTANCE 10

// =====

// SETUP

// =====

void setup()

{

    Serial.begin(115200);

    // Ultrasonic

    pinMode(TRIG_PIN, OUTPUT);

    pinMode(ECHO_PIN, INPUT);

    // LEDs

    pinMode(GREEN_LED, OUTPUT);

    pinMode(RED_LED, OUTPUT);

    // Buzzer
```

```
pinMode(BUZZER, OUTPUT);

// LCD
Wire.begin(SDA_PIN, SCL_PIN);

lcd.init();

lcd.backlight();

// Startup message

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("SMART FLOOD");

lcd.setCursor(0, 1);

lcd.print("MONITORING");

delay(2000);

lcd.clear();

// =====

// CONNECT TO ADAFRUIT IO

// =====

Serial.println();

Serial.println("Connecting to Adafruit IO...");

io.connect();

while (io.status() < AIO_CONNECTED)

{

    Serial.print(".");

    delay(500);

}

Serial.println();

Serial.println("Adafruit IO Connected!");

lcd.clear();

lcd.setCursor(0, 0);
```

```
lcd.print("Adafruit IO");

lcd.setCursor(0, 1);

lcd.print("Connected!");

delay(2000);

lcd.clear();

}

// =====

// GET ULTRASONIC DISTANCE

// =====

float getDistance()

{

    digitalWrite(TRIG_PIN, LOW);

    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);

    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration = pulseIn(ECHO_PIN, HIGH, 30000);

    if (duration == 0)

    {

        return -1;

    }

    float distance = duration * 0.0343 / 2;

    return distance;

}

// =====

// CALCULATE WATER LEVEL

// =====

int calculateWaterLevel(float distance)
```

```

{
  /*
    Maximum safe measurement = 30 cm
    Minimum measurement = 0 cm
    As distance decreases,
    water level percentage increases.
  */
  int level = ((SAFE_DISTANCE - distance) / SAFE_DISTANCE) * 100;
  if (level < 0)
    level = 0;
  if (level > 100)
    level = 100;
  return level;
}

// =====
// MAIN LOOP
// =====

void loop()
{
  // Keep Adafruit IO connection alive
  io.run();

  // Read distance
  float distance = getDistance();

  // Sensor error
  if (distance < 0)
  {
    Serial.println("Ultrasonic sensor error!");
    lcd.clear();
  }
}

```

```
lcd.setCursor(0, 0);  
lcd.print("Sensor Error!");  
lcd.setCursor(0, 1);  
lcd.print("Check HC-SR04");  
digitalWrite(GREEN_LED, LOW);  
digitalWrite(RED_LED, LOW);  
digitalWrite(BUZZER, LOW);  
delay(2000);  
return;  
}  
  
// Calculate water level  
int waterLevel = calculateWaterLevel(distance);  
  
// Variables for status  
String floodStatus;  
  
// =====  
// SAFE CONDITION  
// =====  
  
if (distance > SAFE_DISTANCE)  
{  
    floodStatus = "SAFE";  
    digitalWrite(GREEN_LED, HIGH);  
    digitalWrite(RED_LED, LOW);  
    digitalWrite(BUZZER, LOW);  
    lcd.clear();  
    lcd.setCursor(0, 0);  
    lcd.print("Water:");  
    lcd.print(distance, 1);  
    lcd.print("cm");
```

```

    lcd.setCursor(0, 1);

    lcd.print("STATUS: SAFE");
}

// =====

// WARNING CONDITION

// =====

else if (distance > DANGER_DISTANCE &&
         distance <= SAFE_DISTANCE)
{
    floodStatus = "WARNING";
    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, HIGH);
    // Beeping buzzer
    digitalWrite(BUZZER, HIGH);
    delay(200);
    digitalWrite(BUZZER, LOW);
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Water:");
    lcd.print(distance, 1);
    lcd.print("cm");
    lcd.setCursor(0, 1);
    lcd.print("WARNING!");
}

// =====

// FLOOD CONDITION

// =====

else

```

```
{  
  floodStatus = "FLOOD";  
  digitalWrite(GREEN_LED, LOW);  
  digitalWrite(REDA_LED, HIGH);  
  // Continuous buzzer  
  digitalWrite(BUZZER, HIGH);  
  lcd.clear();  
  lcd.setCursor(0, 0);  
  lcd.print("Water:");  
  lcd.print(distance, 1);  
  lcd.print("cm");  
  lcd.setCursor(0, 1);  
  lcd.print("!!! FLOOD !!!");  
}  
  
// =====  
// SERIAL MONITOR  
// =====  
Serial.println("-----");  
Serial.print("Distance: ");  
Serial.print(distance);  
Serial.println(" cm");  
Serial.print("Water Level: ");  
Serial.print(waterLevel);  
Serial.println(" %");  
Serial.print("Status: ");  
Serial.println(floodStatus);  
  
// =====  
// SEND DATA TO ADAFRUIT IO
```

```
// =====  
waterDistanceFeed->save(distance);  
waterLevelFeed->save(waterLevel);  
floodStatusFeed->save(floodStatus.c_str());  
Serial.println("Data sent to Adafruit IO!");  
Serial.println("-----");  
// Wait before next reading  
delay(5000);  
}
```


9. System Working

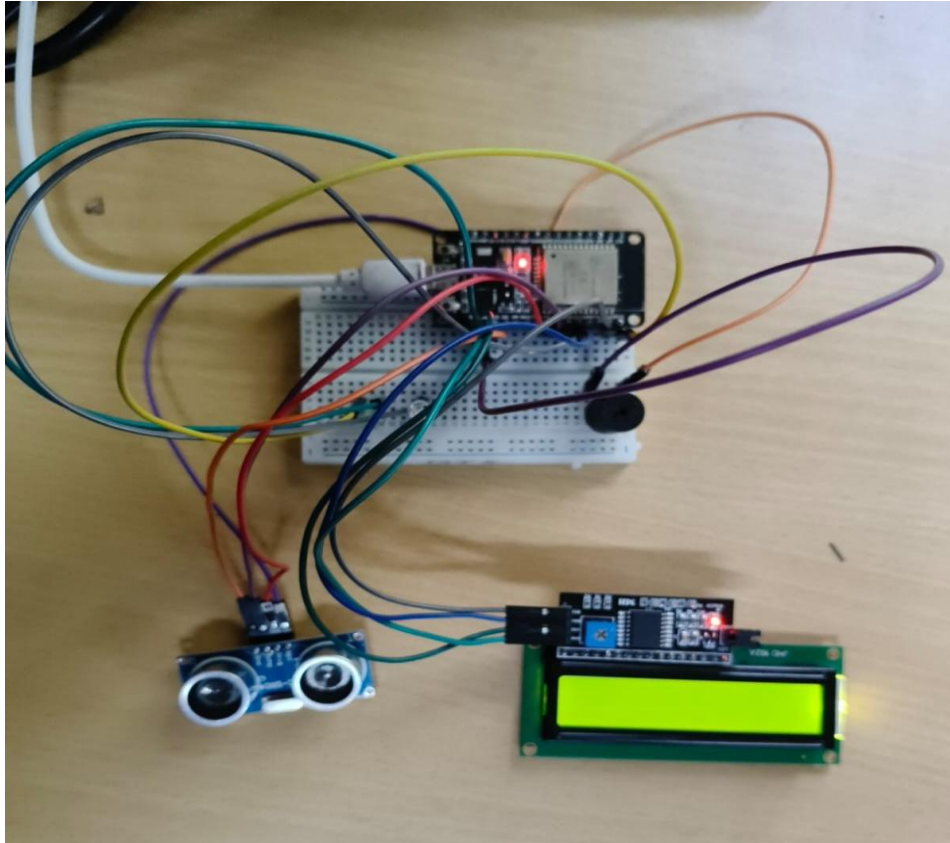
- The ultrasonic sensor continuously measures the distance between the sensor and the water surface.
- The microcontroller calculates the water level based on the measured distance.
- The water level is compared with predefined safe, warning, and critical levels.
- If the level reaches the warning stage, an LED/buzzer provides an alert.
- At the critical flood level, a high-level alarm is activated to warn people.
- The system continuously repeats the measurement for real-time flood monitoring.

10. Bill of Materials

S.No	Component	Quantity
1	Ultrasonic Sensor (HC-SR04)	1
2	Arduino UNO / Microcontroller	1
3	Buzzer	1
4	LED	2-3
5	Resistors	2-3
6	LCD Display (16×2)	1
7	Breadboard	1
8	Jumper Wires	As required
9	Power Supply / USB Cable	1
10	Water Tank/Container for Simulation	1

11. Prototype Implementation

The prototype is developed using an Arduino UNO, ultrasonic sensor, buzzer, LEDs, and LCD display. The ultrasonic sensor is mounted above a water container to measure the water level. The Arduino processes the sensor data and displays the level on the LCD. When the water reaches the warning or critical level, the LED and buzzer are activated to provide an alert. The prototype demonstrates real-time flood level monitoring and early warning.



12. Results & Performance Analysis

The developed system successfully monitored the water level continuously using an ultrasonic sensor. It detected the water level accurately and provided alerts through the LED and buzzer when the water reached the warning or critical level. The prototype showed a quick response and reliable performance, making it useful for early flood detection and disaster management.

Performance Analysis

The system showed good performance in continuously monitoring the water level and detecting changes quickly. The ultrasonic sensor provided reliable measurements, while the microcontroller processed the data and activated alerts at the preset levels. The buzzer and LED responded effectively during critical conditions, demonstrating that the prototype can provide early and reliable flood warnings.



Overall Performance

The smart flood monitoring system performed effectively by continuously measuring water levels and providing timely alerts. The ultrasonic sensor, microcontroller, LED, and buzzer worked together reliably. The system showed quick response, accurate level detection, and effective warning, making it suitable for early flood detection and disaster management applications.